

ASSIGNMENT 03
Due date: 2026/04/[16|14]

**Aim: Mastering Jetpack Compose Basics, State Management,
and Modern Layouts**

1 First Steps with Declarative UI

Jetpack Compose follows a declarative paradigm. Instead of manually finding a view and calling a setter (like `setText()`), you describe the UI based on the current state.

Core Concept: Unidirectional Data Flow

In Compose, state flows *down* (from parent to child) and events flow *up* (from child to parent). When the state changes, the affected Composables are automatically re-executed (**Recomposition**).

☐ Create a new project using the **Empty Activity** template in Android Studio.

☐ **Task: State-driven Greeting**

- Create a Composable function that displays a greeting message (e.g., "Hello, User!").
- Add a `Button` with the label "Change Name" below the text using a `Column`.
- Implement a state variable using `remember { mutableStateOf(...) }`.
- **Interaction:** When clicking the button, cycle the name through a list of names (e.g., "Alice", "Bob", "Compose") or toggle between different messages.

☐ **Deep Dive: The "State Variable" Experiment**

- Try to use a simple local variable (e.g., `var myName = "User"`) without `mutableStateOf` first.
- Observe what happens when you click the button. Explain why the UI does not update even though the variable's value changes in the background.

- **Tooling:** Use the **Layout Inspector** in Android Studio to observe the "Recomposition Count". How many times is your Greeting Composable recomposed when you use the correct state-based approach?

□ **Bonus Challenges:**

- Make the button text change dynamically as well (e.g., showing the next name in the queue).
- Add a Row with a second button to reset the message to its initial state.
- Experiment with changing the text color and background dynamically based on the current state.

2 Modifiers

Objective: Enhance your app by applying Modifiers to control size, padding, background, alignment, and events effectively.

Technical Guide: Modifier & State Snippet

```
// Example: Clickable state-driven Box
var isClicked by remember { mutableStateOf(false) }
Box(modifier = Modifier
    .size(100.dp)
    .background(if (isClicked) Color.Green else Color.Red)
    .clickable { isClicked = !isClicked })
```

- **Button Enhancement:** Apply a specific background color, padding, and rounded corners (`clip` or `shape`).
- **Stateful Logic:** Make the button **disabled** after it has been clicked once.
- **Typography:** Improve the Text with a larger font (`fontSize`), custom color, and a background.
- **Advanced Modifier:** Add a clickable Box with a **dashed border** that changes its color when clicked.
- **Layout Control:** Experiment with `fillMaxWidth()`, `align()`, and `wrapContentSize()` to ensure proper alignment.
- **Research:** Find out which measurement units are supported in Android (`dp`, `sp`, etc.). Which ones should be used for which use case?

Hint: Drawing Dashed Borders

To implement a dashed border, you cannot use the standard `Modifier.border`. Instead, use `Modifier.drawBehind` and the `drawRoundRect` function with a `PathEffect.dashPathEffect`.

3 State in Compose: Unit Converter

- **Objective:** Build a unit converter app (e.g., meter to inch, Celsius to Fahrenheit).
- **UI Elements:**
 - Select a conversion mode using a `DropDownMenu`.
 - Use a `TextField` for input and display the result in a second `TextField` or `Text` element.
 - Use a `Button` to trigger the calculation.
- **State:** Use `remember` and `mutableStateOf` to handle the dynamic updates.
- **Analysis:** Explain *Stateless vs. Stateful* Composables and the concept of *State Hoisting* [2].

4 Dynamic Lists: LazyColumn & Scaffold

In Compose, we use `LazyColumn` to display large lists efficiently. Unlike a standard `Column`, it only renders items currently visible on the screen, similar to the legacy `RecyclerView`.

Technical Guide: Material Scaffold

The `Scaffold` composable provides a standardized Material Design layout structure with slots for a `TopAppBar`, a `FloatingActionButton` (FAB), and the main content area.

- **Data Model:** Define a `data class Message` with properties for *sender*, *text*, *timestamp*, and *type* (Inbound/Outbound).
- **Lazy Loading Requirements:**
 - Generate a static list of **at least 20 to 30 messages** with varying text lengths to ensure the list exceeds the screen height.
 - Display these messages using a `LazyColumn`.
 - **Observation:** Use the **Layout Inspector** while scrolling. Observe how the recomposition count behaves for items entering and leaving the viewport.
- **Custom Styling:** Implement a `ChatBubble` Composable. Use conditional logic to style messages based on the sender:
 - **Inbound:** Aligned to the left, using a specific background color and rounded shape.
 - **Outbound:** Aligned to the right, using a contrasting color.

- **Scaffold Integration:** Wrap your UI in a **Scaffold** and add a **TopAppBar** with the title "Compose Chat".

Bonus Challenge: Interactivity & Legacy Comparison

- **Dynamic Updates:** Use the **Floating Action Button** (FAB) to add a new message to your list dynamically.
- *Hint:* To make a list observable in Compose, use `mutableStateListOf<Message>()` instead of a regular `List`.
- **Reflection:** Briefly compare this implementation to a legacy `RecyclerView`. Which approach do you find more intuitive regarding "Boilerplate" code?

References

- [1] Google. *Thinking in Compose*. <https://developer.android.com/jetpack/compose/mental-model>
- [2] Google. *State and Jetpack Compose*. <https://developer.android.com/jetpack/compose/state>